Group ni Alex

Lunti: Smart Personal Finance Management
Software Requirements Specification
Version 01.00

Revision History

Date	Version	Description	Author
09/05/2026	01.00	Creation of SRS	Atayde, Ygot, Birondo, Enriquez

Table of Contents

1. Introduction 5

1.1 Purpose 5

1.2 Scope 5

1.3 Definitions, Acronyms, and Abbreviations 6

1.4 References 6

1.5 Overview 7

2. Overall Description 7

2.1 Product Perspective 7

2.2 Product Functions 7

2.3 User Classes and Characteristics 8

2.3.1 Regular User 8

2.3.2 Administrator 8

2.4 Operating Environment 8

2.5 Technology Stack 9

2.6 Assumptions and Dependencies 9

2.7 Constraints 9

3. Specific Requirements 10

3.1 Functionality 10

3.1.1 Expense Management 10

3.1.2 Receipt Scanning and OCR 10

3.1.3 Budget Management 11

3.1.4 Spending Analytics 11

3.1.5 Purchase Pattern Tracking 11

3.1.6 Smart Replenishment 11

3.1.7 Smart Shopping List 12

3.1.8 Ingredient Management 12

3.1.9 Recipe Recommendations 12

3.1.10 AI-Powered Insights 12
3.1.11 User Feedback and Personalization 13
3.2 Usability 13
3.2.1 Ease of Use 13
3.2.2 Consistent User Interface 13
3.2.3 Feedback and Confirmation 13
3.2.4 Accessibility 13
3.3 Reliability & Availability 14
3.3.1 Data Reliability 14
3.3.2 Error Recovery 14
3.3.3 Availability 14
3.4 Performance 14
3.5 Security 14
3.5.1 Authentication 14
3.5.2 Authorization 14
3.5.3 Password Security 14
3.5.4 Data Privacy 15
3.5.5 Account Management 15
3.6 Design Constraints 15
3.6.1 Mobile-First Design 15
3.6.2 Data-Dependent Intelligence 15
3.6.3 OCR Limitations 15
3.6.4 AI Limitations 15
3.7 Interfaces 15
3.7.1 User Interface 15
3.7.2 Camera Interface 16
3.7.3 OCR Interface 16
3.7.4 Database Interface 16
3.7.5 Notification Interface 16

1. Introduction

The introduction of the Software Requirements Specification (SRS) provides an overview of the Lunti application, including its purpose, scope, definitions, references, and organization. The aim of this document is to define the functional and non-functional requirements of Lunti and provide a common reference for system design, development, testing, and evaluation.

1.1 Purpose

The purpose of this Software Requirements Specification is to define the functional and non-functional requirements of Lunti, a smart personal finance and grocery management application. Lunti is designed to help users record and monitor expenses, manage budgets, understand spending behavior, track frequently purchased products, receive replenishment reminders, manage grocery and ingredient information, and discover recipes based on ingredients they already have. The system connects financial information with everyday purchasing and food-related behavior to help users make more informed spending decisions, reduce unnecessary purchases, and minimize food waste.

1.2 Scope

Lunti covers personal financial and grocery-related management through a centralized mobile application. Its primary scope includes:

- Personal expense recording and transaction management
- Manual expense entry
- Receipt scanning and information extraction
- Automatic expense categorization
- Flexible daily, weekly, and monthly budgeting
- Budget monitoring and alerts
- Spending analytics and trends
- Purchase pattern tracking
- Frequently purchased product detection
- Smart replenishment reminders
- Smart shopping list management
- Food and ingredient recognition
- Basic grocery and ingredient tracking
- Recipe recommendations based on available ingredients
- Budget-aware meal recommendations
- Personalized spending and purchasing insights
- User feedback for improving recommendations and reminders

The initial system is intended primarily for individual users managing their personal finances and purchases. Advanced features such as bank integration, family/shared accounts, multiple currencies, nutrition tracking, and advanced financial forecasting are outside the initial implementation scope.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
Lunti	The name of the smart personal finance and grocery management application.
SRS	Software Requirements Specification.
OCR	Optical Character Recognition, a technology used to extract text and information from images such as receipts.
AI	Artificial Intelligence, used by Lunti to analyze user data and generate personalized insights and recommendations.
UI	User Interface; the visual elements through which users interact with the application.
UX	User Experience; the overall experience and ease of interaction with the application.
MVP	Minimum Viable Product; the initial version containing the essential features required for Lunti to operate.
RBAC	Role-Based Access Control; a security mechanism that restricts system functions based on user roles.
API	Application Programming Interface; a mechanism that allows different software components to communicate.
Transaction	A recorded financial activity involving an expense or purchase.
Budget	A spending limit established by the user for a specific period or category.
Replenishment	The process of identifying products that may need to be purchased again based on previous purchasing behavior.
Ingredient Inventory	A record of food or ingredients that the user is estimated or confirmed to currently have.
Smart Shopping List	A shopping list generated or enhanced using spending history, purchase patterns, inventory, recipes, and budget information.
Purchase Pattern	A recurring behavior identified from a user's previous purchases, such as frequency, quantity, price, or purchasing interval.

1.4 References

- Lunti Final App Concept & Feature Specification – primary functional and conceptual reference for the system.
- Project proposal and approved system documentation.
- UI/UX design documentation and system wireframes.
- Database design and system architecture documentation.
- Software testing and evaluation documentation.

1.5 Overview

The remaining sections of this document provide a general description of Lunti, including its users, operating environment, major capabilities, assumptions, dependencies, and limitations. Section 3 gives the specific functional and non-functional requirements, including usability, reliability, performance, security, design constraints, and interfaces. The document is intended to provide a common reference for developers, UI/UX designers, testers, project stakeholders, and evaluators.

2. Overall Description

2.1 Product Perspective

Lunti is a mobile-based personal finance and grocery management application that integrates financial tracking with purchasing and food management.

Unlike a conventional expense tracker that primarily records how much a user spends, Lunti analyzes recorded transactions to identify what the user purchases, how frequently purchases occur, when products may need replenishment, and what meals can be prepared using available ingredients.

The system follows the general process:

Scan/Input → Track → Analyze → Learn → Predict → Remind → Plan → Cook → Save

It connects **Money → Purchases → Groceries → Ingredients → Meals**.

2.2 Product Functions

- Expense Tracking
- Receipt Scanning and OCR
- Expense Categorization
- Budget Management
- Spending Analytics
- Purchase Pattern Tracking
- Smart Replenishment
- Smart Shopping List

- Ingredient Management
- Recipe Recommendations
- Budget-Aware Meal Recommendations
- AI-Powered Spending Insights
- Personalized Learning and Feedback

2.3 User Classes and Characteristics

2.3.1 Regular User

The regular user is the primary user of Lunti. The user can create and manage an account, record expenses, scan receipts, review extracted receipt information, create and manage budgets, view spending analytics, track purchases, manage shopping lists, view replenishment reminders, manage ingredients, view recipe recommendations, receive personalized insights, and modify or dismiss recommendations.

The system is intended for individuals such as students, young professionals, and users who want to better manage personal spending and everyday purchases.

2.3.2 Administrator

The administrator manages system-level information and maintenance functions. Administrative responsibilities may include managing user accounts, expense categories, system content, recipe information where applicable, system activity, and application-level configurations. Administrative access must be separated from normal financial data access and should not provide unrestricted visibility into users' private financial information unless explicitly required by the approved requirements.

2.4 Operating Environment

- Lunti is intended to operate primarily as a mobile application.
- Android
- Mobile application frontend
- Backend server/API
- Cloud or hosted database
- OCR processing service
- AI/analytics processing
- Secure cloud storage for receipt images where required

2.5 Technology Stack

Component	**Technology**
Mobile Application	React Native + TypeScript
Backend	Python + FastAPI
Database	PostgreSQL
Authentication	Supabase Auth
Cloud Storage	Supabase Storage
Receipt OCR	Google Cloud Vision API
Artificial Intelligence	OpenAI API
Push Notifications	Firebase Cloud Messaging
State Management	Zustand
API/Data Fetching	TanStack Query
Version Control	Git + GitHub
API Documentation	FastAPI / OpenAPI (Swagger)
Testing	Jest, React Native Testing Library, Pytest
Design/Prototyping	Figma

2.6 Assumptions and Dependencies

- Users have a valid account.
- Users provide accurate information when manually entering expenses.
- Receipt images are sufficiently clear for OCR processing.
- Users review OCR results before saving them as transactions.
- Internet connectivity is available for cloud-dependent functions.
- AI-generated insights depend on the availability and quality of user-recorded data.
- Purchase predictions become more useful as the system collects more historical purchase information.
- Users can correct inaccurate product, category, quantity, or price information.
- The receipt workflow specifically requires user confirmation or correction before extracted information is saved.

2.7 Constraints

- It focuses on personal financial management.
- Advanced banking integrations are not part of the MVP.
- AI recommendations depend on available user data.

- OCR accuracy may vary depending on receipt quality.
- Product recognition may not correctly identify every product.
- Purchase predictions may be unavailable or inaccurate when insufficient historical data exists.
- Ingredient availability may require user confirmation.
- The system should not represent AI predictions as guaranteed financial outcomes.

3. Specific Requirements

The specific requirements are described below.

3.1 Functionality

This subsection contains the functional requirements for Lunti. Requirements are organized by the major features of the application and expressed as system capabilities and ‘shall’ statements where appropriate.

3.1.1 Expense Management

Purpose: Allow users to record and manage financial transactions.

- The system shall allow the user to select Add Expense.
- The system shall allow the user to enter amount, date, merchant, category, and description.
- The system shall validate entered expense information.
- The system shall save confirmed transactions.
- The system shall make saved transactions available in the user's transaction history.
- The system shall update relevant budget and analytics calculations after a transaction is saved.
- The system shall allow users to edit or delete applicable transactions.

3.1.2 Receipt Scanning and OCR

Purpose: Reduce manual entry by extracting purchase information from receipts.

- The system shall allow the user to select Scan Receipt.
- The system shall open the camera or image-selection interface.
- The system shall allow the user to capture or select a receipt image.
- The system shall process the receipt image using OCR.
- The system shall extract available merchant, date/time, product, quantity, price, discount, tax, and total information.
- The system shall display extracted information for user verification.
- The system shall allow the user to correct inaccurate extracted information.
- The system shall require user confirmation before storing the extracted transaction.

3.1.3 Budget Management

Purpose: Allow users to establish and monitor spending limits.

- The system shall support daily budgets.
- The system shall support weekly budgets.
- The system shall support monthly budgets.
- The system shall support custom budget periods.
- The system shall support category-specific budgets.
- The system shall calculate remaining budget based on recorded expenses.
- The system shall notify users when spending approaches, reaches, or exceeds a budget limit.

3.1.4 Spending Analytics

Purpose: Help users understand their spending behavior.

- The system shall provide daily, weekly, and monthly spending summaries.
- The system shall provide category breakdowns.
- The system shall provide spending trends.
- The system shall provide previous-period comparisons.
- The system shall provide recurring expense information.
- The system shall provide product purchasing patterns.

3.1.5 Purchase Pattern Tracking

Purpose: Identify recurring purchasing behavior.

- The system shall analyze number of purchases, last purchase date, average purchasing interval, quantity, product price, purchase frequency, historical spending, and store/merchant information.
- The system shall use historical information to identify recurring purchasing patterns.

3.1.6 Smart Replenishment

Purpose: Predict when regularly purchased products may need to be purchased again.

- The system shall analyze historical product purchases.
- The system shall identify regularly purchased products.
- The system shall estimate the user's purchasing interval.
- The system shall determine whether a product is approaching its expected repurchase period.
- The system shall generate a replenishment suggestion when confidence is sufficient.
- The system shall allow the user to add, dismiss, or modify a suggestion.

- The system should generally avoid automatic replenishment reminders for irregular purchases.

3.1.7 Smart Shopping List

Purpose: Help users organize items they may need to purchase.

- The system shall consider purchase history, replenishment predictions, confirmed inventory, planned meals, recipe requirements, and available budget when generating or enhancing shopping lists.
- The system shall allow users to add items.
- The system shall allow users to remove items.
- The system shall allow users to dismiss suggestions.
- The system shall allow users to mark items as purchased.
- The system shall allow users to manually enter items.

3.1.8 Ingredient Management

Purpose: Help users understand what food ingredients they may currently have.

- The system shall identify food-related products from recorded purchases where possible.
- The system shall distinguish food-related products from non-food household products where possible.
- The system may estimate ingredient availability using purchase dates, quantities, historical consumption, recipe usage, and user confirmation.

3.1.9 Recipe Recommendations

Purpose: Recommend meals based on available ingredients.

- The system shall identify available ingredients.
- The system shall compare available ingredients against recipe requirements.
- The system shall identify missing ingredients.
- The system shall estimate additional purchase cost where applicable.
- The system shall display recipe information.
- The system shall prioritize recipes that use ingredients already available to the user.
- Recipe information may include ingredients, available ingredients, missing ingredients, estimated additional cost, servings, and preparation time.

3.1.10 AI-Powered Insights

Purpose: Convert collected financial and purchasing information into personalized insights.

- The system may analyze spending behavior, purchase patterns, grocery behavior, ingredient information, and budget behavior.

- The system may identify increased food spending, high restaurant spending, average grocery spending, frequently purchased products, and potential upcoming purchases.
- The system should primarily interpret the user's recorded data rather than provide generic advice.

3.1.11 User Feedback and Personalization

Purpose: Use user responses to improve recommendations and reminders.

- The system shall allow users to provide feedback on recommendations.
- The system shall support feedback such as Add to list, I still have some, Bought elsewhere, Don't remind, and Dismiss.
- The system may use feedback to improve future recommendations, purchasing predictions, recipes, and reminders.

3.2 Usability

3.2.1 Ease of Use

Lunti shall provide a simple and understandable interface that allows users to complete common tasks with minimal steps. Frequently used actions, particularly Scan Receipt and Add Expense, should be easily accessible.

3.2.2 Consistent User Interface

The application shall maintain consistent navigation, button placement, icons, typography, colors, terminology, and feedback messages. Users should be able to understand interface elements without extensive instruction.

3.2.3 Feedback and Confirmation

The system shall provide clear feedback when an operation is successful, processing, invalid, or unsuccessful.

3.2.4 Accessibility

- Use readable text
- Maintain sufficient contrast
- Provide understandable icons
- Avoid relying exclusively on color
- Provide clear touch targets
- Support different screen sizes

3.3 Reliability & Availability

3.3.1 Data Reliability

The system shall preserve confirmed transaction and budget data without unintended modification or loss. Financial calculations should be based on stored transaction data and deterministic system logic.

3.3.2 Error Recovery

If receipt processing fails, the system shall allow the user to retry scanning, select another image, or enter the transaction manually. If an internet-dependent request fails, the application should provide an understandable error message and allow the user to retry.

3.3.3 Availability

The application should be available whenever the backend and required external services are operational. Functions that depend on external services, such as OCR or AI processing, may become temporarily unavailable if those services cannot be reached.

3.4 Performance

Lunti shall provide responsive interaction for common operations such as opening screens, navigating between pages, viewing transactions, adding expenses, viewing budgets, and viewing analytics. Receipt scanning and AI processing may require additional processing time. During longer operations, the application shall display a loading or processing state instead of appearing unresponsive. The system should avoid unnecessary repeated processing and should retrieve only the data necessary for the current screen.

3.5 Security

3.5.1 Authentication

Users shall authenticate before accessing their personal financial information. The system shall support secure account authentication and session management.

3.5.2 Authorization

The system shall implement role-based access control. Regular users shall only access their own financial and grocery information. Administrative functions shall only be available to authorized administrators.

3.5.3 Password Security

Passwords shall not be stored as plain text. The authentication system shall use secure password handling and authentication mechanisms.

3.5.4 Data Privacy

Financial, transaction, purchasing, and personal information shall be treated as private user data. The application shall restrict access to data according to the user's authorization.

3.5.5 Account Management

Users should be able to manage their account and request account deactivation or deletion. When an account is deleted, the system should follow the project's defined data-retention and deletion policy.

3.6 Design Constraints

3.6.1 Mobile-First Design

Lunti shall be designed primarily for mobile devices and optimized for touch interaction. The interface should accommodate different screen sizes and orientations where appropriate.

3.6.2 Data-Dependent Intelligence

Lunti's recommendations and predictions depend on the quality and quantity of recorded user data. A new user with little purchase history may not receive reliable replenishment predictions. The system should distinguish between sufficient data, limited data, and insufficient data.

3.6.3 OCR Limitations

Receipt extraction depends on image quality and receipt format. Poor lighting, blurred images, folded receipts, unusual layouts, or unclear text may reduce OCR accuracy. Extracted receipt information must therefore be presented to the user for confirmation before being saved.

3.6.4 AI Limitations

AI-generated insights should be based on available user data and should not be treated as guaranteed predictions. Deterministic calculations such as budget totals and remaining funds should be handled by application logic rather than relying solely on AI interpretation.

3.7 Interfaces

3.7.1 User Interface

The Lunti user interface shall contain the primary sections required for financial and grocery management. Primary navigation may include Home, Transactions, Analytics, Meals, and Budget/Profile. A prominent Scan Receipt action shall be available because receipt scanning is one of the primary methods of recording purchases.

3.7.2 Camera Interface

The system shall interface with the device camera to allow users to capture receipt images. The camera interface should provide receipt framing guidance, a capture button, a retake option, and a confirmation option.

3.7.3 OCR Interface

The OCR component shall receive receipt images and return extracted information. The application shall display extracted information for user verification before creating the final transaction.

3.7.4 Database Interface

The application shall communicate with the backend/database to create records, retrieve records, update records, delete authorized records, store user preferences, and retrieve analytics data.

3.7.5 Notification Interface

The notification system shall provide reminders and alerts for events such as a budget approaching its limit, a budget being exceeded, a replenishment prediction, a shopping reminder, or a meal recommendation. Notifications should be actionable where appropriate, allowing users to navigate directly to the related feature.